

2024



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Московский государственный технический университет имени

Н.Э. Баумана

(национальный исследовательский университет)»

(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1 ПО ДИСЦИПЛИНЕ: ТИПЫ И СТРУКТУРЫ ДАННЫХ

«Длинная арифметика»

Студент: Колоколов Г.И.

Группа: ИУ7-35Б

Название предприятия **НУК ИУ МГТУ им. Н. Э. Баумана**

Студент _____ Колоколов Г. И.

Преподаватель _____ Никульшина Т. А.

2024

Цель работы

Реализовать арифметическую операцию деления над числами, выходящими за разрядную сетку персонального компьютера.

Постановка задачи

Составить программу деления двух чисел, где порядок имеет до 5 знаков: от -99999 до $+99999$, а мантисса – до 40 знаков.

Входные данные

Пользователь вводит два действительных числа: программа выводит “Enter first(или second для второго числа) number:”. Для числа допускается несколько вариантов его ввода: в классической форме и в экспоненциальной.

Классическая форма - число в обычном его представлении, например: “123”

Экспоненциальная форма - число с возможностью указать мантиссу и порядок, например: “0.321e123”

Ограничение длины мантиссы - 40 символов, порядка - 5 символов.

Выходные данные

- 1) Результат вычислений в заданном формате: “Division result: $\pm 0.m1 E \pm K1$ ”, где $m1$ - мантисса длиной до 40 цифр включая, $K1$ - порядок длиной до 5 цифр.
- 2) Сообщение соответствующей ошибки.

Ограничения ввода

- 1) Запрещен ввод пустой строки
- 2) Список разрешенных символов: “0123456789.eE+-”
- 3) Количество значащих цифр в мантиссе не больше 40
- 4) Количество значащих цифр в порядке не больше 5

Задача программы

- 1) Приглашение ко вводу и ввод двух чисел
- 2) Обработка и валидация ввода
 - 3.1) Если введенные данные корректны, то рассчитать результирующее значение
 - 3.2) Если введенные данные не корректны, вывести сообщение об ошибке

Коды завершения программы

Код завершения	Символьное обозначение	Описание кода
0	SUCCESS	Успех
1	IO_ERR	Ошибка ввода

2	MANTISSA_OVERFLOW	Переполнение количества цифр в мантиссе
3	EXPONENT_OVERFLOW	Переполнение количества цифр в порядке
4	EMPTY_NUMBER_ERROR	На вход вместо числа была подана пустая строка
5	WRONG_SYMBOL_ERROR	В числе обнаружались недопустимые символы
6	ZERO_DIVISION	Ошибка деления числа на 0
7	NORMALIZATION_ERROR	Ошибка при нормализации числа

Описание структуры данных

```
#define MAX_MANTISSA_SIZE 40
typedef struct {
    bool sign;
    char mantissa[MAX_MANTISSA_SIZE + 1];
    int exponent;
    size_t point_pos;
} long_number_t;
```

Описание алгоритма

Алгоритм сравнивает мантиссу делимого с мантиссой делителя, если мантисса делителя меньше, то из мантиссы делимого вычитается мантисса делителя. Если мантисса делителя больше, чем мантисса делимого, то мантисса делителя сдвигается на один разряд вправо. Так происходит пока мантисса делимого не станет равна 0 или пока количество цифр в мантиссе результирующего числа не будет превосходить максимального количества цифр в мантиссе. В результате порядки вычитаются, число нормализуется.

Используемые для алгоритма функции

```
int read_long_number(char *number, long_number_t *new_number); // функция
для чтения длинных чисел

void move_str_left(char *data, size_t offset); // функция для сдвига
строки/мантиссы влево на определенное кол-во символов

void trim_mantissa(long_number_t *number); // обрезание нулей мантиссы

int normalize(long_number_t *number); // нормализация числа

bool is_less_divider(char *dividend, char *divider, size_t pos); //
сравнение на то, что делитель меньше мантиссы делимого

void subtract_divider(char *dividend, char *divider, size_t pos); //
вычитание делителя из мантиссы делимого
```

```
void create_bigger_mantissa(char *mantissa, char *new_mantissa, size_t
size); // расширение мантиссы для более точного значения при делении

bool is_zero(char *number); // проверка числа на 0

bool round_int(char *number); // округление числа

int divide(long_number_t *dividend, long_number_t *divider); // алгоритм
деления длинных чисел

void print_long_number(long_number_t *number, bool print_size); // вывод
длинного числа
```

Позитивные тесты

Ввод	Выход	Описание
155e10 155e10	0.1e1	Деление числа на самого себя
255e1 1	0.255e4	Деление числа на 1
0 258e10	0.0e0	Деление 0 на число
1e1 -2e0	-0.5e1	Деление положительного на отрицательное число
-2e0 -1e1	0.2e0	Деление отрицательного на отрицательное
100e99999 1000	0.1e99999	Проверка порядков
1 6	0.1666666666666666666666666667e0	Проверка округления числа
100 37	0.27027027027027027027027027027027e1	Проверка обрезания нулей
100 35	0.28571428571428571428571428571428e1	Проверка округления числа

Негативные тесты

[illegible]

+1-1e1	Error: entered wrong symbol in number!	Некорректный - в мантиссе
1e1+1	Error: entered wrong symbol in number!	Некорректный + в порядке
3e.2	Error: entered wrong symbol in number!	Точка после e
1e99999 0.1	Error: normalization failed!	Результат не может быть нормализован

Выводы по проделанной работе

Для некоторых чисел не хватает встроенных в язык программирования си числовых типов данных, если требуется очень точное или очень длинное число, в таком случае программисту приходится самому реализовывать типы данных для хранения таких чисел и математические операции для этих типов данных

Контрольные вопросы

1. Каков возможный диапазон чисел, представляемых в ПК?

Это зависит от разрядности процессора:

Для 32 битных процессоров - диапазон от -2^{31} до $2^{31}-1$ для знаковых чисел и 2^{32} для беззнаковых чисел

Для 64 битных процессоров - диапазон от -2^{63} до $2^{63}-1$ для знаковых чисел и 2^{64} для беззнаковых чисел

2. Какова возможная точность представления чисел, чем она определяется?

Для числа с плавающей точкой размером 64 бита:

1. Знак числа - 1 бит
2. Экспонента(порядок) - 11 бит
3. Мантисса - 52 бит

Максимальное кол-во знаков в мантиссе - $2^{52} \sim 15-16$ символов => точность числа зависит от количества бит в мантиссе.

3. Какие стандартные операции возможны над числами?

Унарные операции:

- Инкремент (++)
- Декремент (--)

Бинарные операции:

- Сложение (+)
- Вычитание (-)
- Деление (/)
- Умножение (*)
- Взятие остатка (%)

4. Какой тип данных может выбрать программист, если обрабатываемые числа превышают возможный диапазон представления чисел в ПК?

Такой тип данных, в котором можно хранить большие числа посимвольно. В некоторых языках программирования есть встроенные типы данных для работы с такими числами, например BigInteger/BigDecimal в Java.

5. Как можно осуществить операции над числами, выходящими за рамки машинного представления?

Используя обычные для чисел операции, но поразрядно.